

William Jackson

Senior Software Engineer – Angular Python(Django/FastAPI) AWS

Wellington, Nevada, United States | +1 (786) 949-7561 | williamjdev@outlook.com | linkedin.com/in/william-jackson-40036a3ba

PROFESSIONAL SUMMARY

Senior Software Engineer with ~10 years delivering **Python** and **Angular** solutions across data-intensive SaaS, operational analytics, and reliability-focused platform engineering, combining **Python 3.8–3.12**, **Angular 10–18**, and cloud-native delivery into production systems that stay stable under change. Strong record translating ambiguous requirements into measurable releases, modernizing legacy UIs and APIs without breaking downstream consumers, and hardening full-stack platforms through **CI/CD**, **observability**, **security controls**, and incident-driven engineering improvements. Known for solving noisy production issues at the root cause level, improving maintainability through clean service boundaries and reusable frontend architecture, and shipping backend workflows, dashboards, and operational tooling that raise customer trust and reduce support drag.

SKILLS

- **Frontend** — **Angular 10–18**, **TypeScript 4.x–5.x**, JavaScript, RxJS, NgRx, Angular Material, HTML5, CSS3, Tailwind CSS, responsive UI design, component libraries, SPA routing, performance tuning, accessibility, form validation, API integration
- **Backend & APIs** — **Python 3.8–3.12**, FastAPI, Flask, Django, SQL, REST APIs, GraphQL, microservices, background jobs, WebSockets, caching strategies, idempotency, rate limiting, async processing, service boundaries, Bash
- **Data, Messaging & Search** — PostgreSQL, MySQL, SQL Server, Redis, MongoDB, DynamoDB, AWS SQS/SNS, Kafka, RabbitMQ, Elasticsearch, OpenSearch, Redshift, BigQuery
- **Cloud & DevOps** — AWS (**EKS, ECS, EC2, Lambda, API Gateway, S3, CloudFront, RDS, DynamoDB, ElastiCache, EventBridge, Step Functions, IAM**), Azure (**AKS, App Service, Azure SQL, Service Bus, Key Vault, Application Insights**), GCP (**GKE, Cloud Run, Pub/Sub, Cloud Storage, BigQuery**), Docker, Kubernetes, Helm, Argo CD, GitOps, Terraform, CloudFormation, GitHub Actions, GitLab CI, Jenkins, Azure DevOps, CircleCI
- **Observability, Quality & Security** — OpenTelemetry, Datadog, Prometheus, Grafana, ELK/EFK, CloudWatch, structured logging, Jest, Mocha, Chai, Cypress, Playwright, Pact-style API contract tests, OAuth2, OpenID Connect, JWT, Okta, Auth0, AWS Cognito, RBAC/ABAC, audit logging

WORK HISTORY

Senior Software Engineer

February 2021 to December 2025

Viario Networks Inc. - Delaware, United States

- Led delivery of a full-stack modernization program built on **Python 3.10–3.12** services and **Angular 14–18** operational portals, turning fragmented monitoring and manual triage into unified workflows that improved release confidence and reduced support escalation volume by **38%**.
- Designed **API-first** backend contracts and reusable **Angular** data-access patterns with typed models, pagination, and backward-compatible response handling, which kept client upgrades stable while multiple internal teams shipped features in parallel.
- Built and operated a model-inference and decision-support service layer with strict validation, structured error payloads, and response budget enforcement, preventing unpredictable downstream behavior during traffic spikes and noisy operational windows.
- Resolved a persistent duplicate-processing bug across **Kafka**, **RabbitMQ**, and **AWS SQS/SNS** consumers by introducing **idempotency keys**, deduplication logic, and replay-safe retry semantics, cutting reprocessing incidents by more than **45%**.
- Implemented end-to-end observability with **OpenTelemetry**, **Datadog**, **Prometheus**, **Grafana**, and **CloudWatch**, replacing alert floods with actionable SLO-oriented signals that improved on-call diagnosis speed by **50%**.
- Migrated legacy admin screens from older template-driven workflows into **Angular 14–16** component-driven modules with shared guards, interceptors, and RxJS-based state handling, improving maintainability and reducing frontend defect recurrence across releases.

- Standardized data access across **PostgreSQL**, **MySQL**, **SQL Server**, **MongoDB**, and **DynamoDB** by enforcing schema rules, indexing standards, and query review practices that kept performance predictable as platform usage expanded.
- Delivered high-read caching with **Redis** and **ElastiCache**, defining TTL, invalidation, and fallback rules that reduced repeated compute pressure by **32%** while preserving correctness for customer-facing and operator-facing workflows.
- Implemented secure access flows with **OAuth2**, **OpenID Connect**, **JWT**, and **AWS Cognito**, adding **RBAC/ABAC** enforcement and audit logging so sensitive actions remained traceable for internal compliance and incident review.
- Orchestrated long-running platform operations through **EventBridge** and **Step Functions**, using durable state transitions and safe rollback behavior to contain failures that previously caused partial execution drift and operator confusion.
- Deployed services and UI workloads through **EKS**, **ECS**, **EC2**, and **Lambda** behind **API Gateway**, packaging releases with **Docker**, **Kubernetes**, and Helm-based rollout patterns that made production changes easier to reason about.
- Strengthened delivery quality with **Jest**, **Cypress**, **Playwright**, **Mocha**, **Chai**, and **Pact-style** contract tests, converting integration failures and regression-prone UI behaviors into pre-release findings instead of production fire drills.

Software Engineer
Avantia, Inc. - Ohio, United States

October 2018 to January 2021

- Built internal analytics and workflow tooling using **Python 3.8–3.10** backend services and **Angular 10–13** frontend modules, giving operations teams a more reliable way to inspect pipeline health, data quality, and job outcomes without engineering assistance.
- Implemented deterministic preprocessing and transformation pipelines that converted raw operational feeds into training-ready and reporting-ready tables, reducing metric drift during reruns, backfills, and partial reprocessing cycles by over **30%**.
- Improved backend reliability by enforcing strict input validation, typed response contracts, and structured error formats, which made API failure diagnosis faster for frontend consumers and sharply reduced unclear support tickets.
- Refactored brittle ETL sequences into smaller composable services and queue-driven worker steps, shrinking the blast radius of partial failures and making recovery workflows safer during high-volume ingestion periods.
- Introduced **Angular** dashboards with reusable tables, filters, and status indicators that replaced spreadsheet-based monitoring, helping stakeholders identify stuck jobs, missing exports, and schema mismatches much earlier in the release cycle.
- Fixed an intermittent data duplication issue caused by overlapping scheduler triggers and retry behavior by adding queue guards, deduplication checks, and execution markers, which stabilized downstream reporting and customer-facing summaries.
- Implemented pragmatic caching with **Redis** and query optimization patterns that reduced repeated reads and improved latency under spiky access patterns, especially during end-of-cycle processing and internal audit reviews.
- Supported multi-cloud export and analytics integrations across **Azure**, **GCP**, and **AWS** destinations, wiring delivery flows into **BigQuery**, storage endpoints, and service adapters while keeping contracts consistent for consumers.
- Built metrics-oriented dashboards and alerting with **Prometheus**, **Grafana**, and **CloudWatch**, creating faster visibility into pipeline lag, backlog growth, and resource pressure that previously took manual investigation to confirm.
- Standardized local development and CI behavior with **Docker**, **GitHub Actions**, **Jenkins**, and reviewable build steps, reducing environment drift and making frontend and backend changes easier to validate before deployment.
- Delivered incremental platform improvements through demos, tradeoff discussions, and measurable acceptance checks, which kept stakeholders aligned while allowing the team to add new features without destabilizing core workflows.

Software Developer
ArnAmy, Inc. - Florida, United States

March 2016 to September 2018

- Developed **Python 3.6–3.8** ETL jobs that normalized operational and event data across multiple sources, enforcing consistent identifiers and timestamps so reporting, exports, and downstream systems worked from the same trusted baseline.
- Built early predictive scoring utilities and experiment-packaging scripts that allowed shared review beyond a single developer machine, making model exploration and iteration more repeatable for the team's evolving analytics work.
- Created REST endpoints for curated datasets with stable pagination, filtering, and response conventions, which prevented clients from overloading services while preserving backward compatibility during frequent feature expansion.
- Implemented reconciliation routines comparing source-of-truth feeds to derived tables, turning vague "numbers look wrong" complaints into actionable discrepancy reports that shortened debugging cycles and improved business trust.
- Automated recurring backfills and validation flows with protective guardrails that blocked duplicate ingestion, protected summary aggregates, and reduced the risk of silent data corruption during reruns and ad hoc recovery tasks.
- Improved relational database performance through indexing changes, query rewrites, and more disciplined access patterns, keeping response times steady even as data volumes and customer usage increased across the platform.
- Added structured logging, practical error context, and failure annotations to long-running services, making production issues diagnosable from logs and traces instead of relying on local debugging and tribal knowledge.
- Built small but useful **AngularJS to Angular 8–10** transition screens for operational review and data verification, helping the team start modernizing legacy frontend flows without forcing a risky all-at-once rewrite.
- Stabilized brittle third-party and internal integrations with timeouts, retries, and safe execution limits, preventing transient upstream failures from cascading into longer outages and repeated operator intervention.
- Introduced containerized workflows using **Docker** and repeatable deployment scripts, reducing dev-to-test inconsistencies and making it easier to validate new features and bug fixes before handoff.

Junior Software Developer
Centric Tech Inc. - New Jersey, United States

October 2014 to February 2016

- Delivered **Python 2.7–3.x** scripts to clean, validate, and reconcile messy operational datasets, replacing manual spreadsheet-heavy work with repeatable checks that improved consistency and reduced avoidable human error.
- Built internal utilities to standardize identifiers, timestamp formatting, and file validation rules, which prevented downstream mismatches and gave more senior engineers a cleaner foundation for reporting and automation work.
- Improved reliability by adding defensive parsing, explicit failure messages, and boundary checks to small services and scripts, surfacing data issues earlier instead of allowing silent failures to spread through workflows.
- Assisted with service integrations by implementing timeouts, safe retries, and basic connection safeguards, reducing the impact of transient network problems that previously caused avoidable job failures and reruns.
- Supported release validation by comparing outputs to known-good samples, expected row counts, and sanity thresholds, helping the team catch regressions before users noticed inconsistencies in operational results.
- Contributed to early CI discipline through scripted validation steps and lightweight pipeline automation, making team handoffs more consistent and helping reduce "works on my machine" problems during shared development.
- Helped troubleshoot production issues through structured logs, query inspection, and careful reproduction steps, gaining experience in isolating bottlenecks and distinguishing bad input data from real system defects.
- Documented expected inputs, outputs, and operational assumptions for small utilities and jobs, which made future changes easier to review and reduced the maintenance burden on more senior team members.

- Collaborated closely with senior engineers to break vague requests into concrete implementation tasks, building a practical foundation in ownership, estimation, and disciplined delivery that carried into later full-stack roles.

EDUCATION

Bachelor's degree in Computer Science

2010 to 2014

Stanford University Department of Computer Science

- Built strong foundations in algorithms, databases, distributed systems, and applied software engineering, which later supported practical work across Python services, Angular applications, data modeling, and production reliability.